

(https://profile.intra.42.fr)

ESCALA PARA O PROJETO MÓDULO PYTHON (/PROJECTS/PYTHON-MODULE-07)

Você deve avaliar 1 aluno desta equipe.



Repositório Git

`git@vogsphere.42porto.com:vogsphere/intra-uuid-20ecb696-bf10-4c`

Introdução

- Mantenha-se educado, cortês, respeitoso e construtivo durante todo o processo de avaliação. O bem-estar da comunidade depende disso.
- Juntamente com a pessoa (ou grupo) que está sendo avaliada, identifique quaisquer problemas e dedique um tempo para discutir e debater os problemas identificados.
- Você deve considerar que pode haver diferenças na forma como seus colegas entenderam as instruções do projeto e o escopo de sua funcionalidade. Mantenha sempre a mente aberta e avalie o trabalho deles com a maior honestidade possível. A pedagogia só é válida se a avaliação por pares for conduzida com seriedade.

Diretrizes

- Avalie apenas os trabalhos que estiverem no repositório Git do aluno ou do grupo.
- Verifique novamente se o repositório Git pertence ao aluno ou ao grupo. Certifique-se de que o trabalho seja referente ao projeto em questão e verifique também se o comando "git clone" foi executado em uma pasta vazia.
- Verifique cuidadosamente se nenhum pseudônimo malicioso foi usado para enganar-lo e levá-lo a avaliar algo diferente do conteúdo do repositório oficial.
- Para evitar surpresas, verifique cuidadosamente se tanto o aluno avaliador quanto o avaliado revisaram quaisquer scripts que possam ser usados para facilitar a avaliação.
- Se o aluno avaliador ainda não tiver concluído o projeto em questão, é obrigatório que ele leia todo o documento da disciplina antes de iniciar a defesa.
- Use os indicadores disponíveis nesta escala para sinalizar um repositório vazio, um programa que não funciona, um erro de norma, fraude, etc. Nesses casos, a avaliação termina e a nota final é 0 (ou -42 em caso de fraude). No entanto, com exceção de casos de fraude, você é encorajado a continuar discutindo seu trabalho (mesmo que não o tenha terminado) para identificar quaisquer problemas que possam ter causado a falha e evitar repetir os mesmos erros no futuro.
- Lembre-se de que, durante a defesa, não é permitido o encerramento inesperado, prematuro ou descontrolado do programa; caso contrário, a nota final será zero. Use a flag apropriada. Você nunca deve editar nenhum arquivo além do arquivo de configuração, se houver. Se precisar editar um arquivo, discuta os motivos com o aluno avaliado e certifique-se de que ambos concordam com a alteração.

Anexos

 assunto.pdf (<https://cdn.intra.42.fr/pdf/pdf/203338/en.subject.pdf>)

Preliminares

Noções básicas

- Avalie apenas os trabalhos que estiverem no repositório Git do aluno.
- Certifique-se de que o repositório Git pertença ao aluno.
- Verifique se não estão sendo usados aliases maliciosos.
- Verifique se a estrutura do projeto corresponde aos requisitos da disciplina: ex0, ex1 e ex2 e um arquivo **.py de inicialização**.

Todas as verificações preliminares foram concluídas com sucesso?

☒ Sim

☐ Não

Exercícios

Exercício 0 - Fábrica de Criaturas

Verifique o conteúdo da pasta ex0/:

- Contém um arquivo **.py de inicialização** e pelo menos mais um arquivo, mas vários arquivos.
- O código define a classe abstrata `Creature`, que herda de `ABC`. O método `attack()` deve ser abstrato e o método `describe()` deve ser um método concreto.
- Pelo menos quatro classes concretas diferentes do tipo `Creature` são definidas, herdando de `Creature`.
- O código define a classe abstrata `CreatureFactory`, que herda de `ABC`. Os métodos abstrato `create_base()` e `create_evolved()` existem.
- pelo menos duas classes de fábrica de concreto, uma para cada família com duas Criaturas.
- O arquivo **init.py** deve expor apenas as fábricas de `Creature`, e não as próprias `Creatures`.

Nota: o tema sugere nomes de criaturas e nomes de fábricas. O projeto avaliado pode usar nomes diferentes, desde que os padrões de projeto envolvidos sejam implementados corretamente. Esta nota é válida apenas se a defesa.

Verifique se o script "battle.py" está presente e segue o cenário estabelecido:

- Ele instancia ambas as classes de fábrica `Creature`.
- Existe uma única função que aceita qualquer fábrica, e ela aplica este processo simples: cria a base, descreve-a e ataca-a, depois repetir o processo com a criatura evoluída.
- Essa função funciona com ambas as fábricas criadas anteriormente.
- Outra função que recebe duas fábricas existe e cria duas criaturas (básicas ou evoluídas), a faz lutar.

Se tudo funcionar corretamente e o padrão `Abstract Factory` estiver implementado corretamente, no caso contrário, marque "Não".

☒ Sim

☐ Não

Exercício 1 - Capacidades

Verifique o conteúdo da pasta ex1/:

- Contém um arquivo **.py de inicialização** e pelo menos mais um arquivo, mas vários arquivos.
- O código define as classes abstratas `HealCapability` e `TransformCapability`, que herdam de `ABC` e contêm os métodos abstratos `heal()` e `transform()` + `revert()`, respectivamente. Para os entus `Creatures`, outras capacidades e métodos associados são permitidos.
- Pelo menos quatro novas classes concretas de Criatura são definidas, herdando da classe `Creature` e implementando uma das duas classes de capacidade acima (duas Criaturas com capacidade de cura e duas com capacidade de transformação).
- pelo menos duas classes de fábrica de concreto, uma para cada nova família com duas Criaturas.
- O arquivo **init.py** deve expor apenas as fábricas de `Creature`, e não as próprias `Creatures`.

Verifique se o script "capacitor.py" está presente e segue o cenário estabelecido:

- Ele instancia ambas as novas classes de fábrica `Creature`.
- Cada fábrica é testada: os métodos padrão `describe()` e `attack()` são usados, juntamente com o método especial inserido no local correto do processo (consulte o assunto para obter informações sobre as capacidades sugeridas). Isso se aplica tanto à Criatura base quanto à Criatura evoluída.

Se tudo funcionar corretamente e as capacidades abstratas estiverem implementadas corretament
Caso contrário, marque "Não".

✓ Sim

✗ Não

Exercício 2 - Estratégia Abstrata

Verifique o conteúdo da pasta ex2/:

- Contém um arquivo .py **de inicialização** e pelo menos mais um arquivo, mas vários arquivo
- O código define a classe abstrata BattleStrategy, que herda de ABC. Os métodos act() e is_ métodos abstratos.
- Pelo menos três classes de estratégia concretas diferentes são definidas, herdando da class
- Existe uma estratégia "normal" que é adequada para qualquer Criatura, independentemente capacidade.
- Existem diferentes estratégias que correspondem às capacidades do ex1. Por exemplo, uma defensiva pode funcionar com uma criatura com capacidade de cura e então chamar heal() ;

Verifique se o script "tournament.py" está presente e segue o cenário estabelecido:

- Ele instancia classes de fábrica Creature de ex0 e ex1.
- Ele instancia as diferentes estratégias disponíveis.
- Existe uma única função battle() que gerencia o torneio (você pode criar subfunções).
- A função battle() aceita, como parâmetro, uma lista de tuplas composta por uma CreatureFa BattleStrategy.
- A função battle() organiza uma luta entre cada par de oponentes da lista. Verifica se uma cri consigo mesma e se não luta duas vezes contra outra criatura (AB e BA).
- Verifique se a estratégia associada é aplicada à criatura durante o combate. Por exemplo, u cura irá curar() ao usar uma estratégia defensiva, mas não ao usar uma estratégia normal.
- Verifique se uma estratégia inadequada para uma criatura interrompe o torneio.

Se tudo funcionar corretamente e o padrão de estratégia abstrata estiver implementado corretamei "Sim". Caso contrário, marque "Não".

✓ Sim

✗ Não

Qualidade e compreensão do código

Qualidade e compreensão do código

Avalie a qualidade e os padrões gerais do código:

- O código foi escrito em Python 3.10 ou superior?
- O código está em conformidade com os padrões do linter flake8 (sem erros)?
- As dicas de tipo são OBRIGATÓRIAS para todas as funções e métodos; verifique-as usando
- As docstrings NÃO são obrigatórias para este módulo.
- O código está bem estruturado e é legível?

Entendimento:

- O código demonstra o uso e a compreensão adequados de padrões abstratos?

Responda "Sim" se todos os padrões de qualidade forem atendidos e "Não" se algum deles estiver

✓ Sim

✗ Não

Classificações

Não se esqueça de verificar a bandeira correspondente à defesa.

✓ OK

★ Projeto excepcional

Trabalho vazio

📁 Trabalho incompleto

⚙️ Compilação inválida

📖 Norma

📝 Trair

⬆️ Situação relativa

💧 Vazamentos

🚫 Função proibida

💬 Não consigo dar suporte/exp

Conclusão

Deixe um comentário sobre esta avaliação (máximo de 2048 caracteres)

Avaliação final

Termos gerais de uso da API
(<https://profile.intra.42.fr/legal/terms/33>)

Declaração sobre o uso de cookies
(<https://profile.intra.42.fr/legal/terms/2>)

Política de Privacidade
(<https://profile.intra.42.fr/legal/terms/5>)

Termos gerais de utilização do site
(<https://profile.intra.42.fr/legal/terms/6>)

Regras de proc
(<https://profile.intra.42>)